Berner Fachhochschule
Technik und Informatik

Enterprise JavaBeans 3.0

Stephan Fischli

This script is based on the book
"Enterprise JavaBeans" by Bill Burke and Richard Monson-Haefel

Contents

Introduction

Enterprise JavaBeans Defined

Sun:

The Enterprise JavaBeans architecture is a component architecture for the development and deployment of component-based distributed business applications. Applications written using the Enterprise JavaBeans architecture are scalable, transactional, and multi-user secure. These applications may be written once, and then deployed on any server platform that support the Enterprise JavaBeans specification.

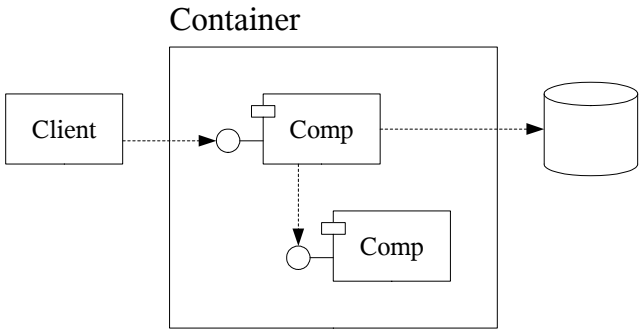
In short:

Enterprise JavaBeans is a standard server-side component model for distributed business applications.

Server-Side Components

Server-side components

- are independent units of executable software
- encapsulate business logic (business objects)
- have well-defined interfaces and context dependencies
- can be assembled to business applications
- are managed by a runtime environment (container)
- are accessible by remote clients (distributed objects)



EJB Roles

- Enterprise Bean Provider
  defines the bean's interfaces and classes
- Application Assembler
  combines enterprise beans to an application
- Deployer
  deploys the enterprise beans into the EJB container
- EJB Server Provider
  provides infrastructure for distributed objects and transactions
- EJB Container Provider
  provides runtime support and deployment tools
- System Administrator
  manages the EJB server and container

History

Mar 1998	EJB Specification 1.0
Dec 1999	EJB Specification 1.1 XML deployment descriptors
Aug 2001	EJB Specification 2.0 Local interfaces Message-driven beans Entity relationships and EJB query language
Nov 2003	EJB Specification 2.1 Support for Web services Timer service
May 2006	EJB Specification 3.0 Metadata annotations Injection mechanism Light-weight persistence layer

References

- Sun (http://java.sun.com/javaee)
  - Java EE Platform and EJB Specification
  - Java EE Software Development Kit
  - Java EE Tutorial
  - Java EE BluePrints
- Bill Burke, Richard Monson-Haefel
  Enterprise JavaBeans, 5th Edition
  O'Reilly, 2006
- Rima Patel Sriganesh, Gerald Brose, Micah Silverman
  Mastering Enterprise JavaBeans 3.0
  John Wiley & Sons, 2006
- Debu Panda, Reza Rahman, Derek Lane
  EJB 3 in Action
  Manning, 2007

Application Servers

- BEA WebLogic Server
  http://www.beasys.com
- IBM WebSphere Application Server
  http://www.software.ibm.com
- Borland AppServer
  http://www.borland.com
- Sun Java System Application Server
  http://java.sun.com
- JBoss
  http://www.jboss.org
- GlassFish
  https://glassfish.dev.java.net

Architectural Overview

Entities

Entities

- model real-world objects (e.g. cabins, customer, ships)
- have persistent state
- are plain old Java object (POJO)
- can be allocated, serialized and sent over the network
- are stored and retrieved through an entity manager

Enterprise Bean Components

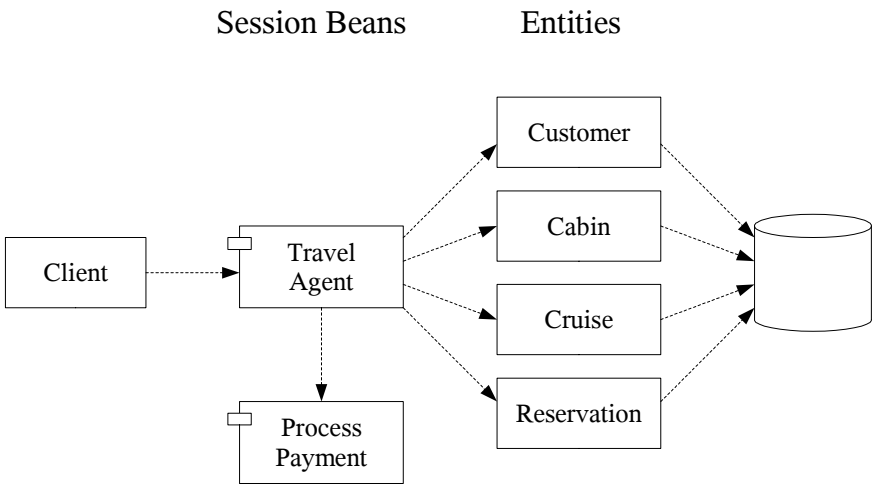
Session Beans

- implement a process or a task (e.g. making a reservation on a cruise)
- are stateless or stateful (conversational state)
- are transient but can access persistent data through entities
- are accessed through a remote interface

Message-driven Beans

- have the same purpose as stateless session beans
- are accessed through asynchronous messages
- are integration points for legacy systems

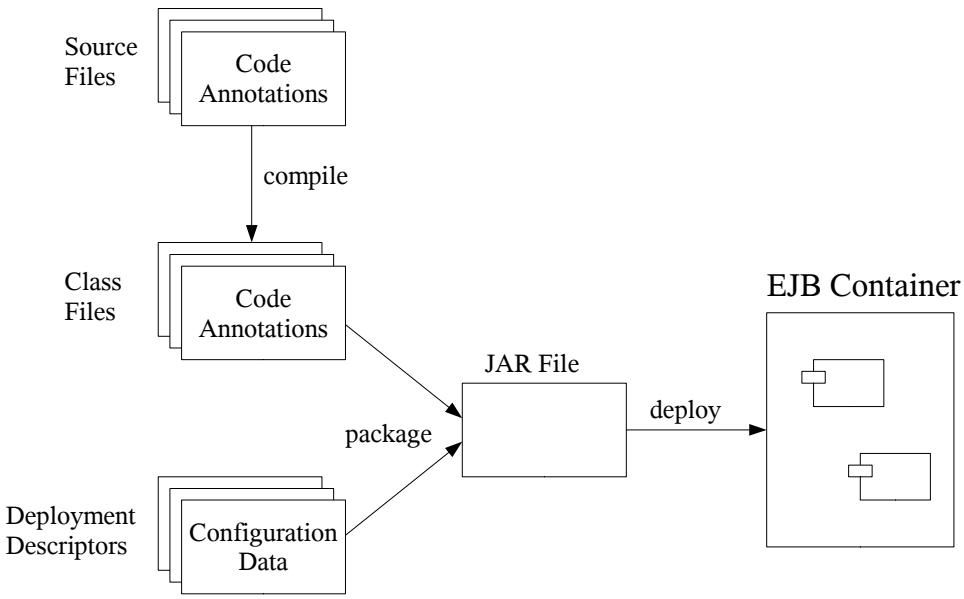
Using Enterprise Beans and Entities



Classes and Interfaces

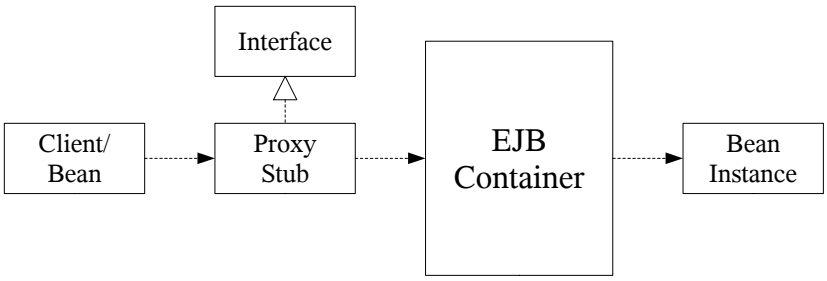
- Remote interface
  defines methods that can be accessed from remote clients
- Local interface
  defines methods that can be used by other beans in the same EJB container
- Endpoint interface
  defines methods that can be accessed from outside the EJB container via the SOAP protocol
- Message interface
  defines methods by which messaging systems can deliver messages asynchronously
- Bean class
  implements the business logic and contains annotations to configure the primary services

Annotations, Deployment Descriptors and JAR Files



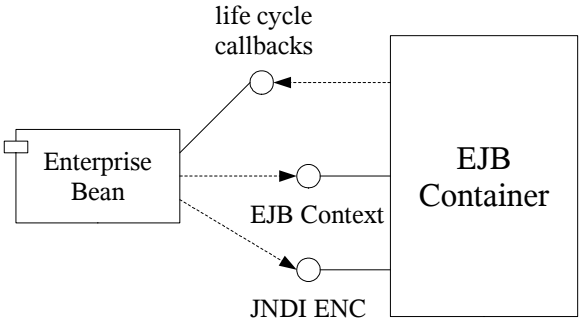
The EJB Container

- An enterprise bean is always invoked through a proxy stub which sends the method call to the EJB container
- The EJB container manages the bean's life cycle, applies the primary services and routes the invocation to an actual instance



The Bean-Container Contract

- Beans can register callback methods for life cycle events that the EJB container emits
- The EJB context provides the bean with information about its environment
- The JNDI environment naming context (JNDI ENC) can be used by the bean to look up resources and other beans



Resource Management and Primary Services

Resource Management

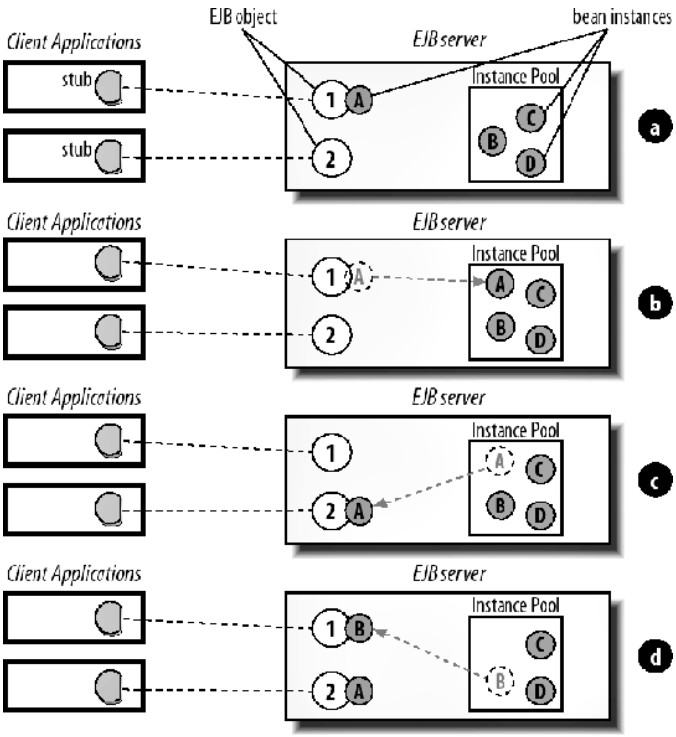
EJB servers increase performance by sharing resources through

- instance pooling
- passivation/activation
- connection management

Instance Pooling

- Instance pooling is used for stateless session and message-driven beans
- The EJB container creates several equivalent instances of a bean and holds them in a pool
- The instances are assigned to EJB requests and JMS messages as needed and returned to the pool after process completion
- A few instances can serve hundreds of clients

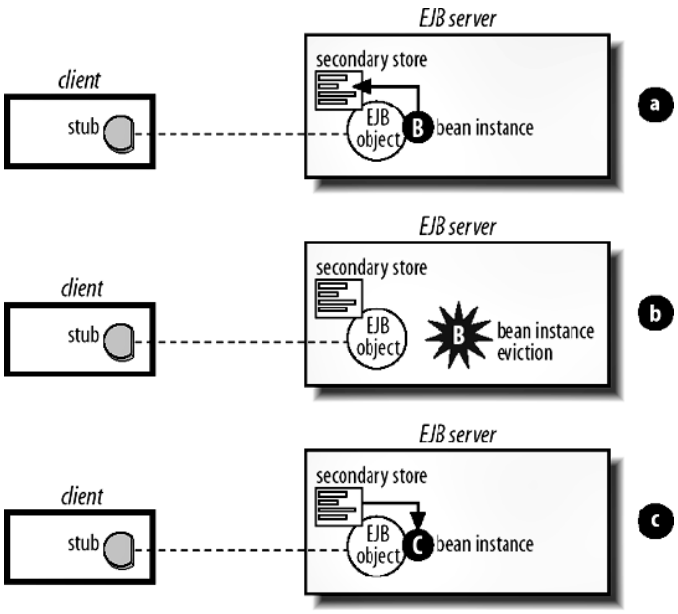
Instance Pooling (cont.)



The Activation Mechanism

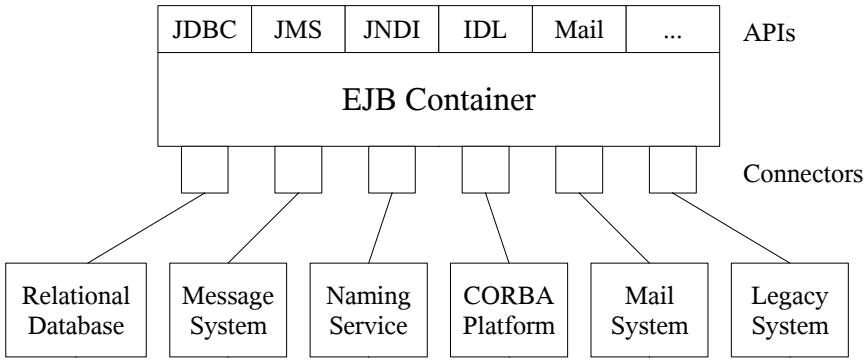
- The activation mechanism is used for stateful session beans
- The EJB container can save the conversational state of a bean to secondary storage and evict the instance from memory (passivation)
- A new instance is created and populated with the initial state when a method on the bean is invoked

The Activation Mechanism (cont.)



The Java EE Connector Architecture (JCA)

- JCA defines an interface between the EJB container and enterprise information system (EIS)
- The EJB container is responsible to pool and maintain the EIS connections



Primary Services

EJB servers automatically manage the primary services:

- Concurrency
- Transactions
- Persistence
- Distributed Objects
- Asynchronous Messaging
- Timer Service
- Naming
- Security

Concurrency

- The EJB specification prohibits to create threads or to use synchronization primitives
- Requests and messages are concurrently processed by multiple instances of the same bean
- Entities are created per transaction and represent a snapshot of the persistent data
- Version fields, isolation levels or explicit locking can be used to prevent corrupt data

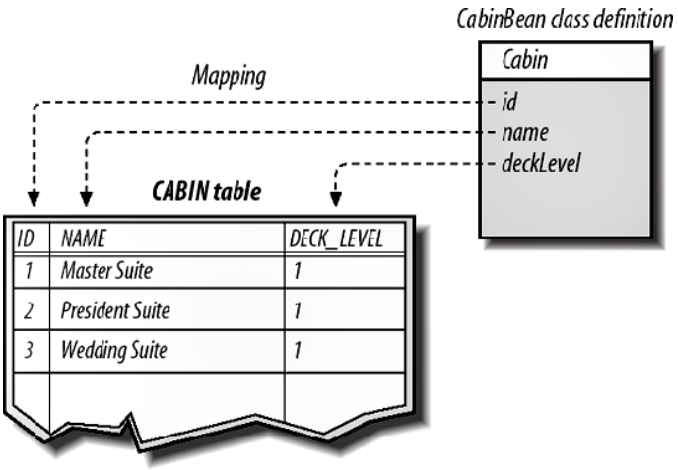
Transactions

- Transactions are managed automatically by the EJB container using transactional attributes of the bean methods
- Transactions can also be managed explicitly through the Java Transaction API (JTA)

Persistence

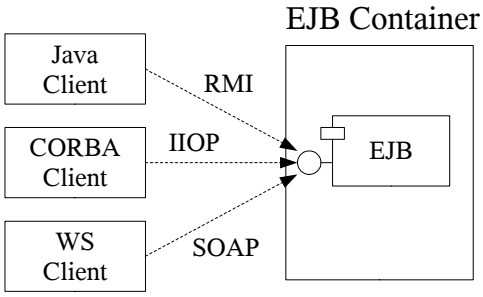
- Java Persistence provides a rich object-to-relational mapping including inheritance, multi-table mappings, versioning and querying support
- The entity manager allows to create, find, remove and update entities
- When an entity is attached, the entity manager automatically synchronizes its state with the data source
- When an entity is detached, it can be used as a plain Java object

Persistence (cont.)



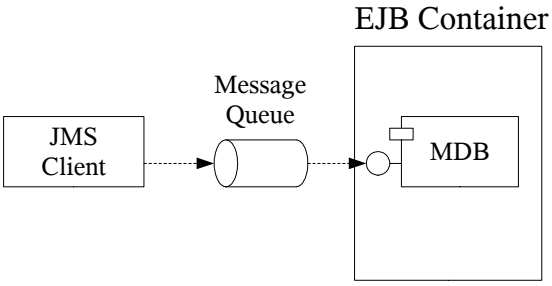
Distributed Objects

- EJB servers are required to support the RMI-IIOP and the SOAP protocol and Java clients using the EJB client API
- EJB servers may also support any other distributed object protocols and clients written in other languages than Java



Asynchronous Enterprise Messaging

- The EJB container reliably routes messages from JMS clients to message-driven beans
- The messages may be persistent such that they survive system failures
- Enterprise messaging is transactional

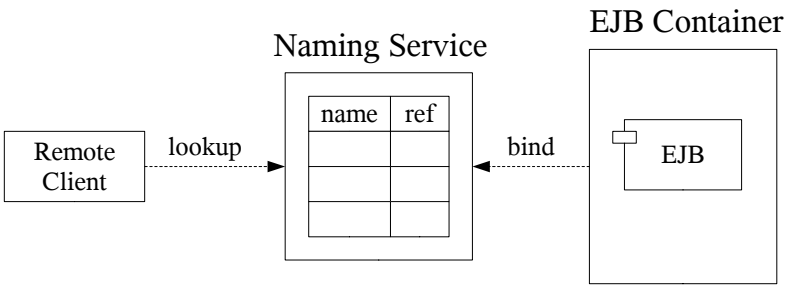


EJB Timer Service

- The EJB timer service allows to schedule notifications that are sent to enterprise beans at specific times
- Timers can be used for self-auditing and batch processing

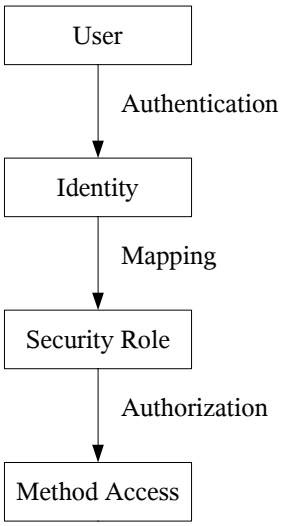
Naming

- A naming service provides clients with a mechanism for locating distributed objects and resources
- EJB servers are required to support the Java Naming and Directory Interface (JNDI) and the CORBA Naming Service



Security

- EJB servers support secure communication, authentication and authorization
- EJB authorization is defined per bean method based on security roles



Persistence: Entity Manager

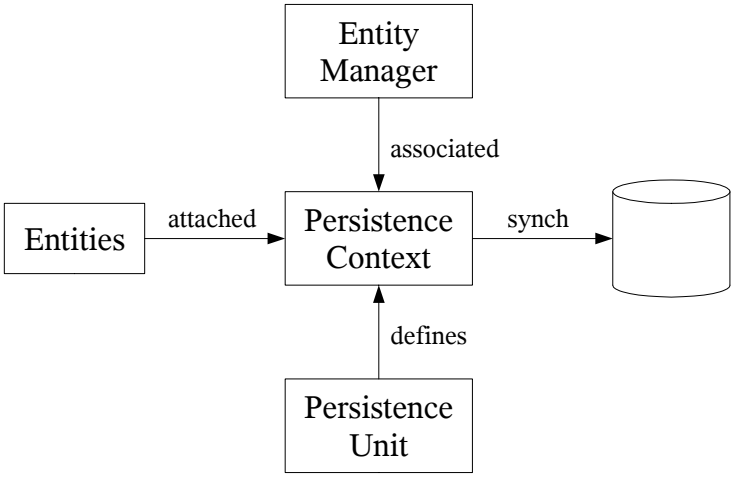
Introduction

Java Persistence

- has its own specification separated from the EJB specification
- provides a light-weight abstraction layer on top of JDBC
- implements an object-to-relational mapping engine
- provides an extended query language
- integrates tightly with the EJB container
- can also be used in plain Java programs

Overview

- Entities are lightweight persistent objects
- A persistence context is a set of entity instances whose life cycles are managed by an associated entity manager
- A persistence unit maps a set of entity classes to a particular database



Entities

Entities

- are plain old Java objects which are annotated with O/R mapping metadata
- have fields or properties which represent their persistent state
- must have a primary key

```
@Entity
public class Customer {
   private int id;
   private String name;

   @Id @GeneratedValue
   public int getId() { return id; }
   public void setId(int id) { this.id = id; }
   String getName() { return name; }
   public void setName(String name) { this.name = name; }
}
```

Entities (cont.)

Entities

- are allocated with the new operator
- become persistent when they are attached to a persistence context using an entity manager

```
Customer customer = new Customer();
customer.setName("Bill");
entityManager.persist(customer);
...
customer = entityManager.find(Customer.class, 1);
entityManager.remove(customer);
```


Managed versus Unmanaged Entities

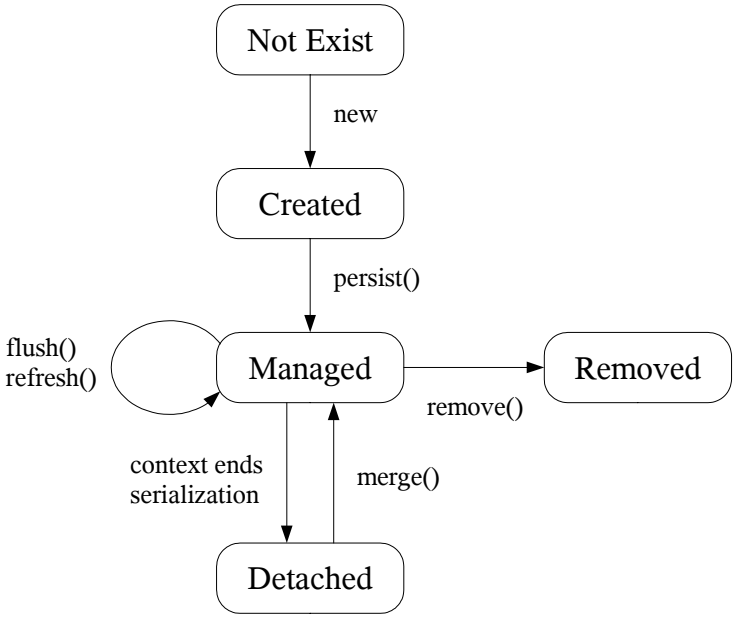
Managed (attached) entities:

- have a persistent identity and belong to a persistence context
- state changes are tracked by an entity manager
- are synchronized with the database when the entity manager interacts with a transaction

Unmanaged (detached) entities:

- have a persistent identity but are not associated with a persistence context
- are not synchronized with the database
- can be serialized and sent across the network to a remote client

Life Cycle of an Entity



Entity Manager

An entity manager manages the entities of a persistence context:

- performs all persistence operations
- provides automatic synchronization and caching
- is responsible for the O/R mapping

Application-Managed Entity Manager

- In a Java SE application, an EntityManagerFactory is used to create an EntityManager
- The Persistence class is responsible for bootstrapping an EntityManagerFactory corresponding to a persistence unit

```
EntityManagerFactory factory =
   Persistence.createEntityManagerFactory("titan");
EntityManager entityManager = factory.createEntityManager();
...
entityManager.close();
```


Container-Managed Entity Manager

- In a Java EE environment, an EntityManager can be injected into an enterprise bean using the @PersistenceContext annotation
- An injected EntityManager must not be closed

```
@Stateless
public class MySessionBean implements MySessionRemote {
   @PersistenceContext(unitName="titan")
   private EntityManager entityManager;
   ...
}
```

Interacting with an Entity Manager

The EntityManager provides methods to

- create, read, update and delete entities (CRUD)
- locate entities by EJB QL or SQL queries
- synchronize the state of entities with the database
- acquire locks on entities

Entity Manager Interface

```
public interface EntityManager {
    public void persist(Object entity);
    public <T> T find(Class <T> entityClass, Object primaryKey);
    public <T> T getReference(Class <T> entityClass, Object primaryKey);
    public <T> T merge(T entity);
    public void remove(Object entity);

    public Query createQuery(String queryString);
    public Query createNamedQuery(String name);
    public Query createNativeQuery(String sqlString);

    public void flush();
    public void refresh(Object entity);
    public void lock(Object entity, LockModeType lockMode);

    public boolean contains(Object entity);
    public void clear();
    public void close();
}
```

Persistence Context

- A persistence context is a set of entity instances which are managed by an entity manager
- The entity manager tracks any state changes and synchronizes them when the transaction is committed, a flush is requested or a correlated query is executed
- When the persistence context is closed, the entity instances become detached

Transaction-Scoped Persistence Context

A transaction-scoped persistence context

- lives as long as a transaction and is automatically closed when the transaction completes
- is only available in a container environment

```
@Stateless
public MySessionBean implements MySessionRemote {
   @PersistenceContext(unitName="titan", type=TRANSACTION)
   EntityManager entityManager;

   @TransactionAttribute(REQUIRED)
   public int createCustomer(String name) {
      Customer customer = new Customer();
      customer.setName(name);
      entityManager.persist(customer);
      return customer.getId();
   }
}
```

Extended Persistence Context

An extended persistence context

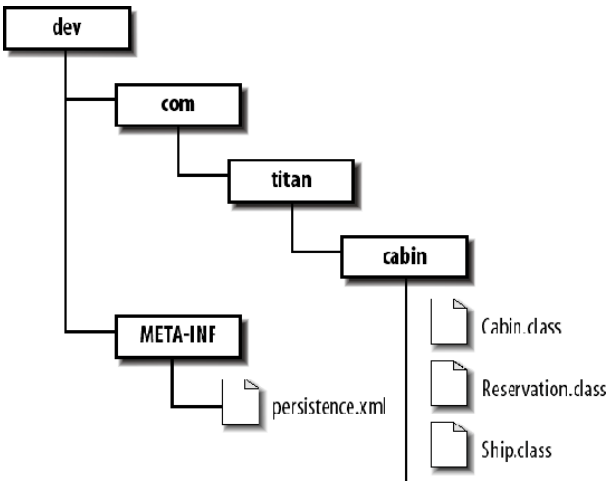
- keeps entity instances managed even after a transaction completes
- is useful in conversational situations because no long-running transactions are needed
- can be used by standalone applications and stateful session beans

```
EntityManagerFactory factory =
   Persistence.createEntityManagerFactory("titan");
EntityManager entityManager = factory.createEntityManager();
EntityTransaction transaction = entityManager.getTransaction();
transaction.begin();
Customer customer = new Customer();
entityManager.persist(customer);
transaction.commit();
...
transaction.begin();
customer.setName("Bill");
transaction.commit();
```


Persistence Unit

A persistence unit

- defines a set of entity classes and maps it to a particular database
- is packaged in a Java archive which contains the entity classes, the persistence deployment descriptor and additional O/R mapping files



Persistence Deployment Descriptor

The persistence deployment descriptor defines the

- name of the persistence unit
- transaction type (JTA or resource local)
- JNDI name of the data source (Java EE)
- class name of the persistence provider (Java SE)
- vendor-specific properties
- names of optional O/R mapping files
- JAR files containing additional entity classes

```
<persistence>
   <persistence-unit name="titan">
      <jta-data-source>java:/OracleDS</jta-data-source>
      <properties>
         <property name="org.hibernate.hbm2ddl" value="update"/>
      </properties>
   </persistence-unit>
</persistence>
```

Session Beans

Introduction

Session beans

- implement tasks and interactions between other beans (workflow)
- may be stateless or stateful
- do not represent persistent data but can access data in the database

The Stateless Session Bean

Stateless session beans

- maintain no state between method invocations (all information has to be passed as method parameters)
- represent collections of related services (e.g. making payments)
- are very efficient since they can be swapped between clients
- may reference generic resources obtained from the JNDI ENC, the session context or by injection

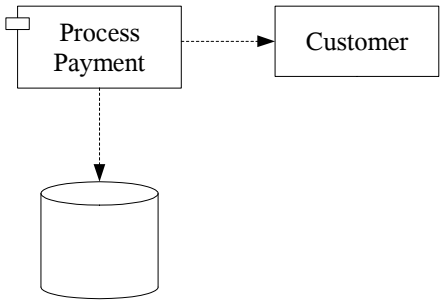
The Business Interface

A session bean has one or more business interfaces

- remote interface:
  method invocations from remote clients
  parameter and return values are copied (call-by-value)
- local interface:
  methods available only within the same EJB container
  parameter and return values are passed by reference

Example: The ProcessPayment EJB

The ProcessPayment EJB is a stateless session bean which is used to charge a customer for the price of a ship cruise



The Business Interface

```
@Remote
public interface ProcessPayment {
   public boolean byCheck(Customer customer, CheckDO check,
      double amount) throws PaymentException;
   public boolean byCash(Customer customer, double amount)
      throws PaymentException;
   public boolean byCredit(Customer customer, CreditCardDO card,
      double amount) throws PaymentException;
}

@Local
public interface ProcessPaymentLocal {
   ...
}
```

The Bean Class

The bean class

- is identified by the @Stateless annotation
- implements its remote and local interfaces
- must have a default constructor

```
@Stateless
public class ProcessPaymentBean
    implements ProcessPaymentRemote, ProcessPaymentLocal {
    ...
}
```


Exceptions

- Application exceptions describe a business problem and should be meaningful to the client
- System exceptions describe a resource problem or a programming error
- An EJBException indicates that the EJB container ran into problems
- RuntimeExceptions are caught by the EJB container and wrapped in an EJBException
- Checked exceptions from subsystems (e.g. NamingException, SQLException) should be caught and rethrown as EJBException

Accessing Environment Properties

- The enterprise naming context (ENC) is used to store configuration data and resource references
- When a bean is deployed, the ENC is populated from annotations and the XML deployment descriptor
- When a bean is instantiated, the EJB container initializes the resource references with values from the ENC (injection)

```
@Stateless
public class ProcessPaymentBean implements ... {
   @Resource(mappedName="titanDB")
   DataSource dataSource;
   @Resource(name="min")
   int minCheckNumber;
   ...
}
```


Session Context

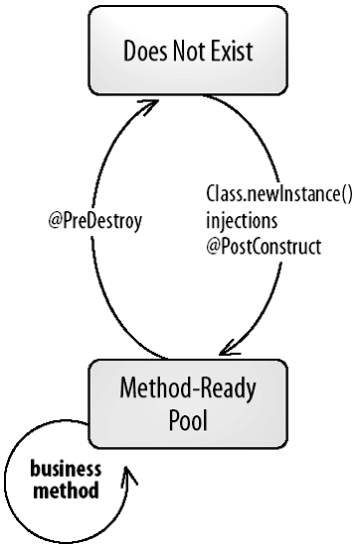
- The session context can be used to get information about a method invocation and provides access to services
- A session bean can obtain its session context by injection

```
@Stateless
public class ProcessPaymentBean implements ... {
   @Resource SessionContext context;
   ...
}
```

Session Context (cont.)

```
public interface SessionContext extends EJBContext {
    <T> getBusinessObject(Class<T> businessInterface);
    Class getInvokedBusinessInterface();
    MessageContext getMessageContext();
}

public interface EJBContext {
    public Object lookup(String name);
    public TimerService getTimerService();
    public Principal getCallerPrincipal( );
    public boolean isCallerInRole(String roleName);
    public UserTransaction getUserTransaction();
    public boolean getRollbackOnly();
    public void setRollbackOnly();
}
```


The Life Cycle of a Stateless Session Bean



Life Cycle Events

- Whenever the EJB container changes the state of a session bean, it triggers corresponding events
- The bean can register callback methods for these events

```
@Stateless
public class MySessionBean implements MySessionRemote {
   @PostConstruct
   public void init() { ... }
   @PreDestroy
   public void cleanup() { ... }
```

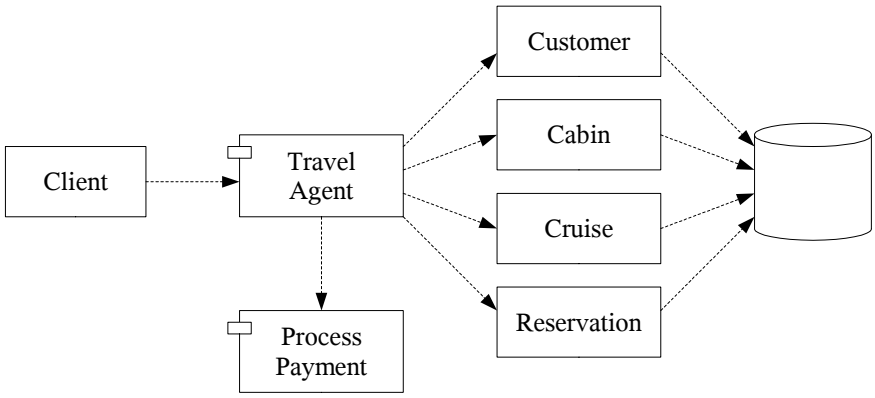

The Stateful Session Bean

Stateful session beans

- are dedicated to one client for the life time of a bean instance
- maintain conversational state between method invocations
- act as agents for the client managing processes and workflows (e.g. making a reservation on a cruise)

Example: The TravelAgent EJB

The TravelAgent EJB is a stateful session bean which implements the process of making a reservation on a ship cruise



The Business Interface

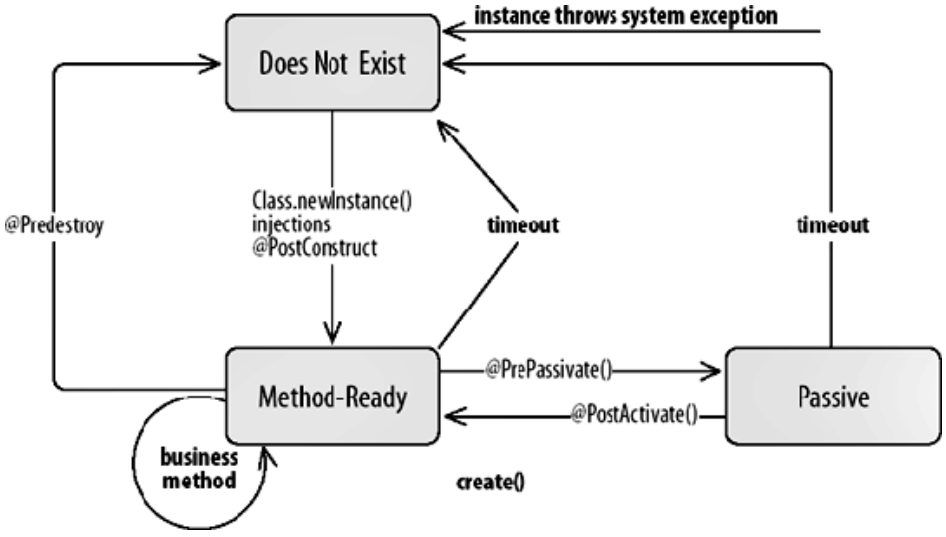
```
@Remote
public interface TravelAgentRemote {
   public Customer findOrCreateCustomer(String first, String last);
   public void updateAddress(Address addr);
   public void setCruiseID(int cruise);
   public void setCabinID(int cabin);
   public TicketDO bookPassage(CreditCardDO card, double price)
      throws IncompleteConversationalState;
}
```

EJB References

- If a session bean needs to access another bean, the @EJB annotation can be used to get a reference to the other bean

```
@Stateful
public class TravelAgentBean implements TravelAgentRemote {
   @EJB
   ProcessPaymentLocal processPayment;
   ...
}
```


The Life Cycle of a Stateful Session Bean



Passivation and Activation

- During passivation the conversational state of a bean instance is preserved by the EJB container:
  - primitive values
  - serializable objects
  - SessionContext, EntityManager, UserTransaction
  - references to other beans
  - managed resource factories
- Fields of other types (e.g. loggers) should be set to null in the @PrePassivate method and re-initialized in the @PostActivate method

Destruction

A stateful session bean is destroyed when

- the client invokes a method annotated as @Remove
- the bean instance times out
- a business method throws a system exception

```
@Stateful
public class TravelAgentBean implements TravelAgentRemote {
   ...
   @Remove
   public TicketDO bookPassage(CreditCardDO card, double price) {
      ...
   }
}
```

Extended Persistence Context

- Stateful session beans are allowed to inject an extended persistence context
- With an extended persistence context, entities remain managed across method invocations
- The persistence context is cleaned up when the bean is removed

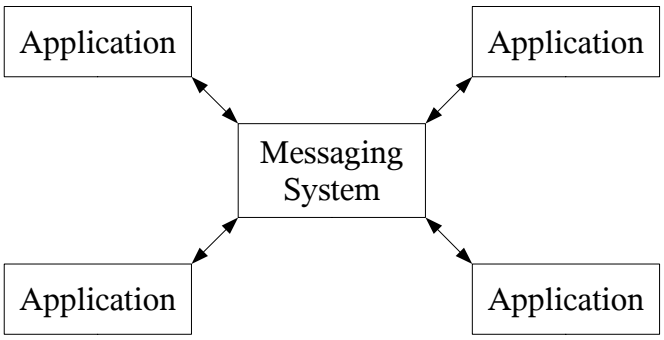
```
@Stateful
public class TravelAgentBean implements TravelAgentRemote {
   @PersistenceContext(unitName="titan", type=EXTENDED)
   EntityManager entityManager;
   ...
}
```

Message-Driven Beans

Messaging Systems

Messaging systems

- facilitate the exchange of messages between applications
- provide fault tolerance, load balancing, scalability, and transactional support



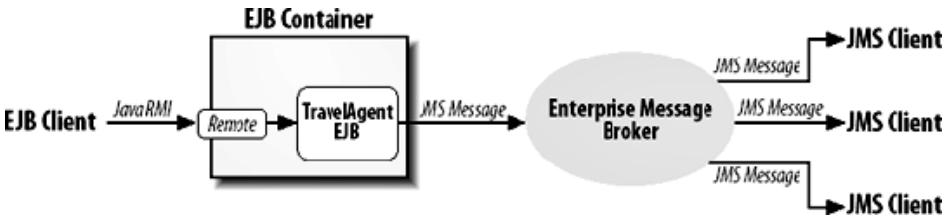
JMS as a Resource

- JMS is a standardized API to access messaging systems
- All EJB vendors must support a JMS provider

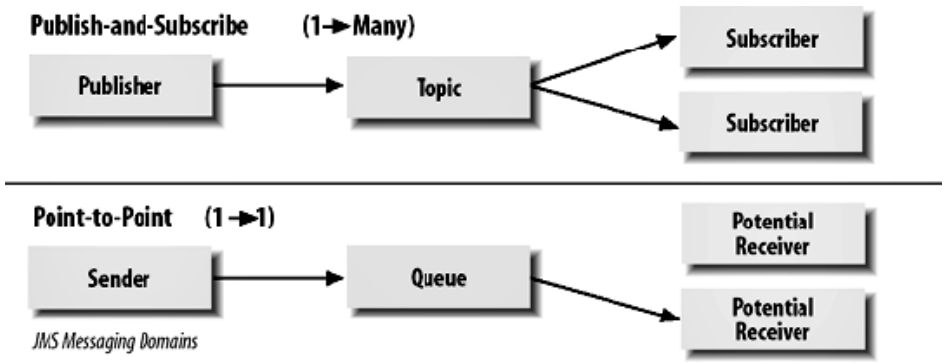
EJB Application			
Java Message Service			
IBM MQSeries	BEA WebLogic	Progress SonicMQ	…

JMS is Asynchronous

- A JMS client (producer) can send a message without having to wait for a reply
- Messages are sent to a destination from which other clients (consumers) can receive them
- Producers are not dependent on the availability of the consumers
- The transaction and security contexts are not propagated



JMS Messaging Models



JMS Messaging Models (cont.)

Publish-and-Subscribe (Topic)

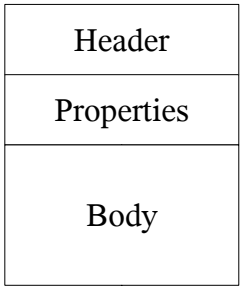
- one producer can send a message to many consumers
- messages are automatically broadcast to the consumers (push model)
- consumers can establish durable subscriptions

Point-to-Point (Queue)

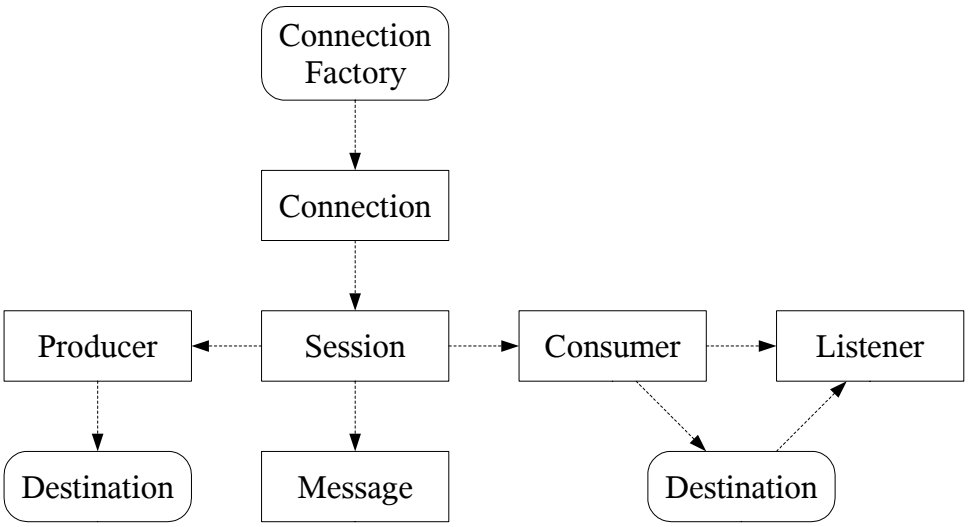
- a producer sends a message to exactly one consumer
- messages are explicitly requested by the consumers (pull model)

JMS Messages

- A JMS message is a Java object which consists of three parts:
  - the header contains delivery information and metadata
  - the properties are application or vendor specific metadata
  - the body carries the application data
- The message type determines the format of the body (text, map, object, stream, bytes)

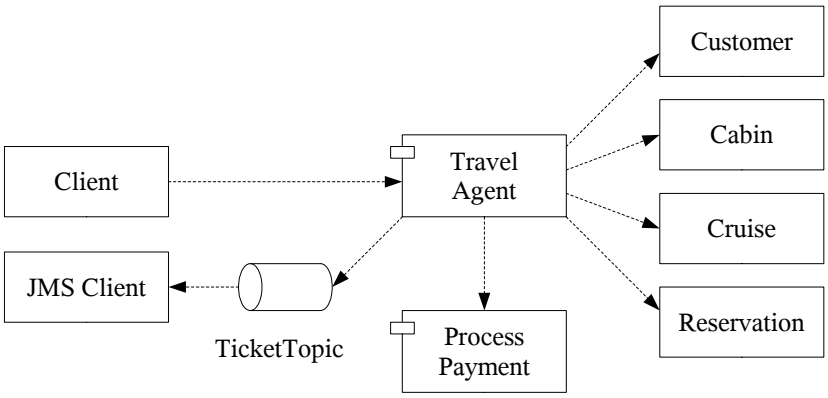


Programming Model



Example: The TravelAgent EJB

The TravelAgent EJB uses JMS to alert some other Java application that a reservation has been made



Connection Factory and Destination

- A connection factory creates connections to the JMS provider
- A destination is a virtual channel through which messages are exchanged
- Connection factories and destinations are administered objects which may be obtained by injection

```
@Stateful
public class TravelAgentBean implements TravelAgentRemote {
   @Resource(mappedName="ConnectionFactory")
   ConnectionFactory connectionFactory;
   @Resource(mappedName="TicketTopic")
   Topic topic;
   ...
}
```

Session

A session

- groups the sending and receiving of messages
- provides a single-threaded context
- manages transactions and controls the acknowledgment

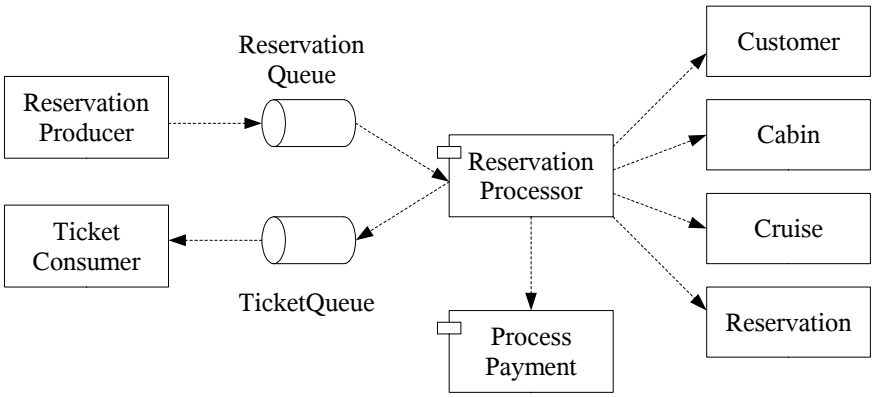
```
Connection connection =
   connectionFactory.createConnection();
Session session = connection.createSession(true, 0);
MessageProducer producer = session.createProducer(topic);
```


JMS-Based Message-Driven Beans

- Message-driven beans are stateless, server-side, transaction-aware components for processing asynchronous JMS messages
- The EJB container manages the bean's environment including transactions, security, resources, concurrency and acknowledgment

Example: The ReservationProcessor EJB

The ReservationProcessor EJB is a message-driven bean which receives JMS messages notifying it of new reservations



The Bean Class

- Message-driven beans implement the MessageListener interface
- The EJB container passes messages to the onMessage() method which contains all the business logic
- Message-driven beans have access to the JNDI ENC and a context object

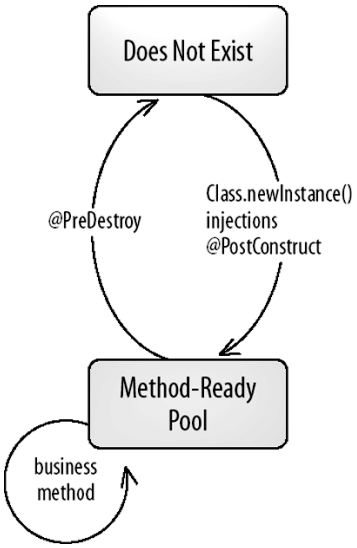
```
@MessageDriven(...)
public class ReservationProcessorBean
   implements MessageListener {
   @Resource MessageDrivenContext context;

   public void onMessage(Message message) {
      ...
   }
}
```

The @MessageDriven Annotation

- The @MessageDriven annotation defines a set of properties to configure the message provider
- JMS properties include the destination type, message selectors, the acknowledgment mode and the subscription durability

```
@MessageDriven(
   activationConfig={
      @ActivationConfigProperty(
         propertyName="destinationType",
         propertyValue="javax.jms.Queue"), ...})
```


The Life Cycle of a Message-Driven Bean



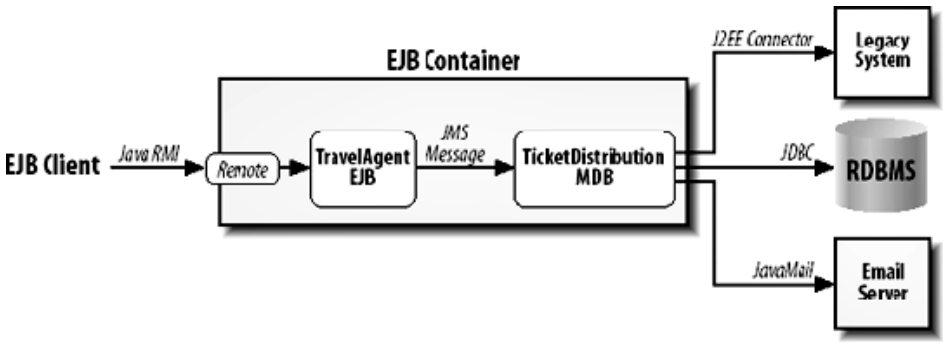
Connector-Based Message-Driven Beans

- Message-driven beans may service any kind of messaging system
- The JCA defines the contract between the EJB container and an asynchronous connector to an EIS
- Message-driven beans can process incoming messages from the EIS by implementing a specific messaging interface

Message-Linking

Message-linking

- allows to route messages from an enterprise bean directly to a message-driven bean
- must be specified in the XML deployment descriptor
- can be implemented by the application server itself



Timer Service

Introduction

- Scheduling systems are used to run programs at specified times e.g. for reporting, auditing or batch processing
- The timer service allows to schedule timers associated with a stateless session or a message-driven bean
- A client must call some method or send a JMS message in order for a timer to be set

Timer Service API

- An enterprise bean schedules itself for a timed notification using the timer service
- When the scheduled time is reached, the EJB container invokes the bean's @Timeout method

```
@Stateless
public class ShipMaintenanceBean implements ShipMaintenance {
   @Resource TimerService timerService;
   public void scheduleMaintenance(String ship, Date date) {
      String info = ship + " is scheduled maintenance";
      timerService.createTimer(date, info);
   }
   @Timeout
   public void maintenance(Timer timer) {
      String info = (String)timer.getInfo();
      ...
   }
}
```


The TimerService Interface

- The timer service allows to create single-action or interval timers
- Timers are made persistent and survive system failures
- When a timer is created in the scope of a transaction which is rolled back, the timer is canceled

```
public interface TimerService {
   public Timer createTimer(Date expiration, Serializable info);
   public Timer createTimer(long duration, Serializable info);
   public Timer createTimer(
      Date initialExpiration, long intervalDuration, Serializable info);
   public Timer createTimer(
      long initialDuration, long intervalDuration, Serializable info);
   public Collection getTimers();
}
```

The Timer

- A Timer object represents a timed event which has been scheduled
- It can be used to cancel the timer, obtain the application data and find out when the timer expires

```
public interface Timer {
    public void cancel();
    public Serializable getInfo();
    public Date getNextTimeout();
    public long getTimeRemaining();
}
```


Problems with the Timer Service

- The timer service is not flexible enough for many real-world scheduling needs (e.g. first Monday of every month)
- It is not possible to program a stop date
- There is no standard way to configure a timer at deploy time
- The Timer object conveys too little information about the timer

Interceptors

Introduction

- Interceptors are objects that intercept method calls or life cycle events of session and message-driven beans
- Interceptors allow to encapsulate common behavior without polluting the business logic

Interceptor Class

- An interceptor class is a class with an interceptor method
- An interceptor method is annotated with the @AroundInvoke annotation and has a single parameter of type InvocationContext

```
public class Profiler {
   @AroundInvoke
   public Object profile(InvocationContext invocation)
      throws Exception {
      long startTimer = System.currentMillis();
      try { return invocation.proceed(); }
      finally {
         long time = System.currentMillis() - startTime;
         System.out.println(invocation.getMethod() + ": " + time + "ms");
      }
   }
}
```


Invocation Context

- The InvocationContext provides information on the business method the client is invoking such as the target bean instance and the actual parameters
- The proceed() method calls the next interceptor if any, otherwise the bean method is invoked

```
public interface InvocationContext {
   public Object getTarget();
   public Method getMethod();
   public Object[] getParameters();
   public void setParameters(Object[] args);
   public Map<String, Object> getContextData();
   public Object proceed() throws Exception;
}
```

Applying Interceptors

- The @Interceptors annotation applies interceptors to an individual bean method or to all methods of a bean if used on the bean class
- The @ExcludeClassInterceptors annotation turns off any class-level interceptors for a particular method

```
@Stateful
@Interceptors(...)
public class TravelAgentBean implement TravelAgentRemote {
   @Interceptors(Profiler.class)
   public TicketDO bookPassage(CreditCardDO card, double price)
      throws IncompleteConversationalState {
      ...
   }
   @ExcludeClassInterceptors
   public void setCustomer(Customer customer) { ... }
}
```


Applying Interceptors through XML

- The use of an XML deployment descriptor allows to configure interceptors at deploy time
- Even default interceptors can be defined which are applied to all beans of a deployment unit

```
<ejb-jar version="3.0" ...>
   <assembly-descriptor>
      <interceptor-binding>
         <ejb-name>*</ejb-name>
         <interceptor-class>com.titan.Profiler</interceptor-class>
      </interceptor-binding>
   </assembly-descriptor>
</ejb-jar>
```

Features of Interceptors

- Interceptors can also be used to intercept EJB life cycle events, e.g. for initializing the state of a bean
- Interceptors have the same life cycle as the bean they intercept
- Interceptors belong to the same JNDI ENC as the bean they intercept and have support of all the injection annotations
- Interceptors are invoked within the normal call stack and thus can use exception handling to control the outcome of a bean method

Transactions

ACID Transactions

A transaction is the execution of several tasks which is required to be

- Atomic:
  A transaction must execute completely or not at all
- Consistent:
  The integrity of the underlying data store is guaranteed
- Isolated:
  A transaction is executed without interference from other transactions
- Durable:
  The data changes are written to physical storage

Transaction Scope

- A transaction is started when a bean's method begins and ends when the method completes
- A transaction is propagated to the invoked beans and the entity manager of the accessed entities
- The transaction scope includes all the enterprise beans and entities that participate in a particular transaction

Transaction Attributes

- Whether an enterprise bean participates in a transaction depends on its transaction attributes
- Transaction attributes can be set for the entire bean or for individual methods by the @TransactionAttribute annotation
- The default attribute is Required

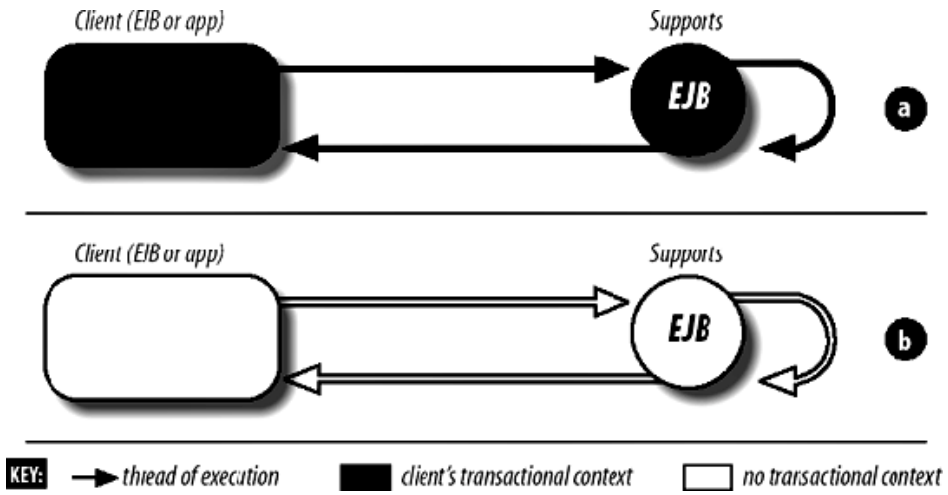
```
@Stateless
@TransactionAttribute(NOT_SUPPORTED)
public class TravelAgentBean implements TravelAgentRemote {
   public void setCustomer(Customer customer) { ... }
   @TransactionAttribute(REQUIRED)
   public TicketDO bookPassage(CreditCardDO card, double price)
   { ... }
}
```

The NotSupported Attribute

The transaction is suspended until the bean method completes

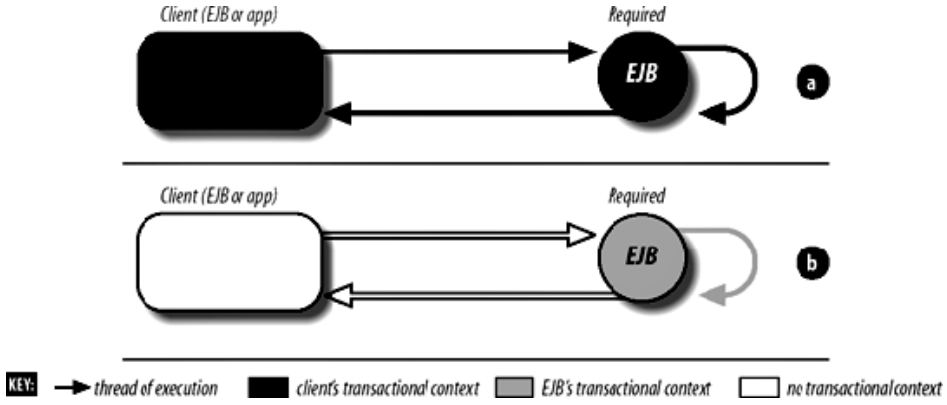
The Supports Attribute

The bean method is included in the scope of an existing transaction



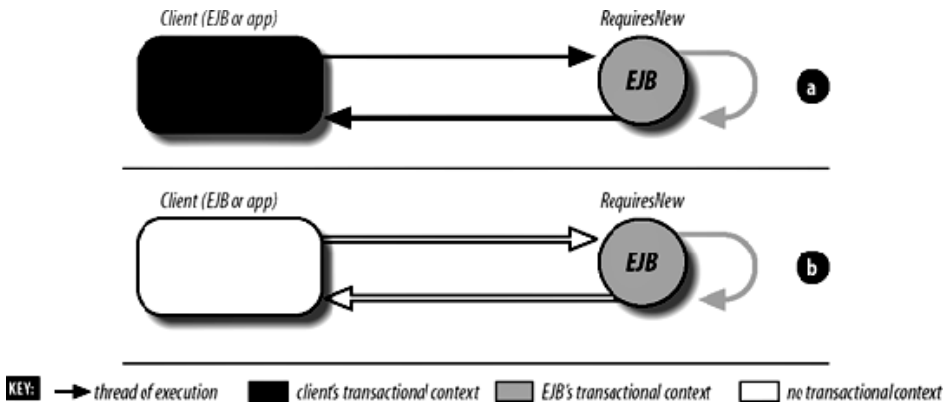
The Required Attribute

The bean method is executed within an existing or a new transaction



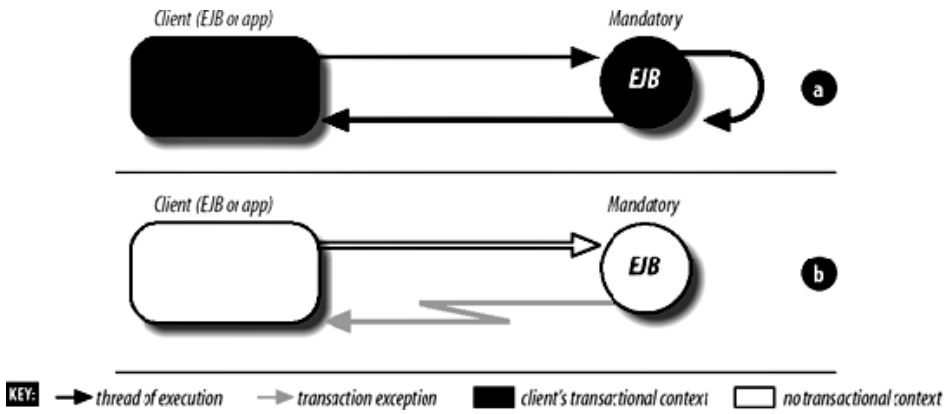
The RequiresNew Attribute

A new transaction is started when the bean method is invoked



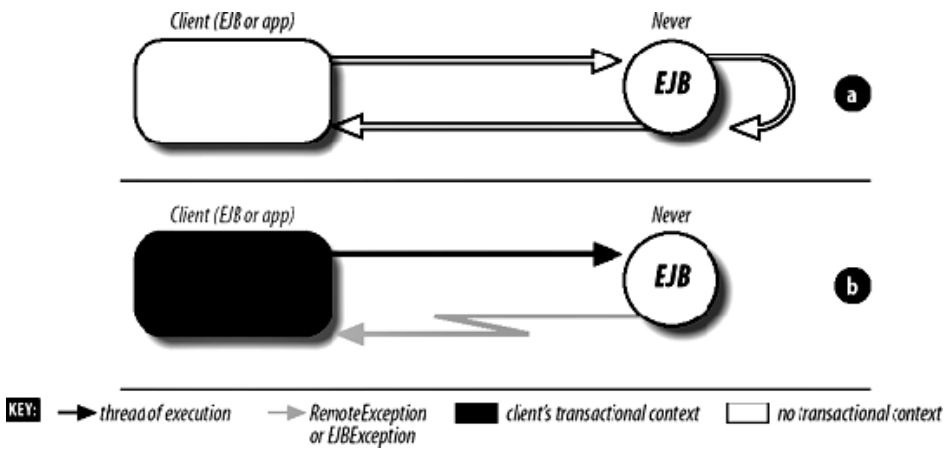
The Mandatory Attribute

The bean method must be invoked within an existing transaction



The Never Attribute

The bean method must not be invoked within a transaction



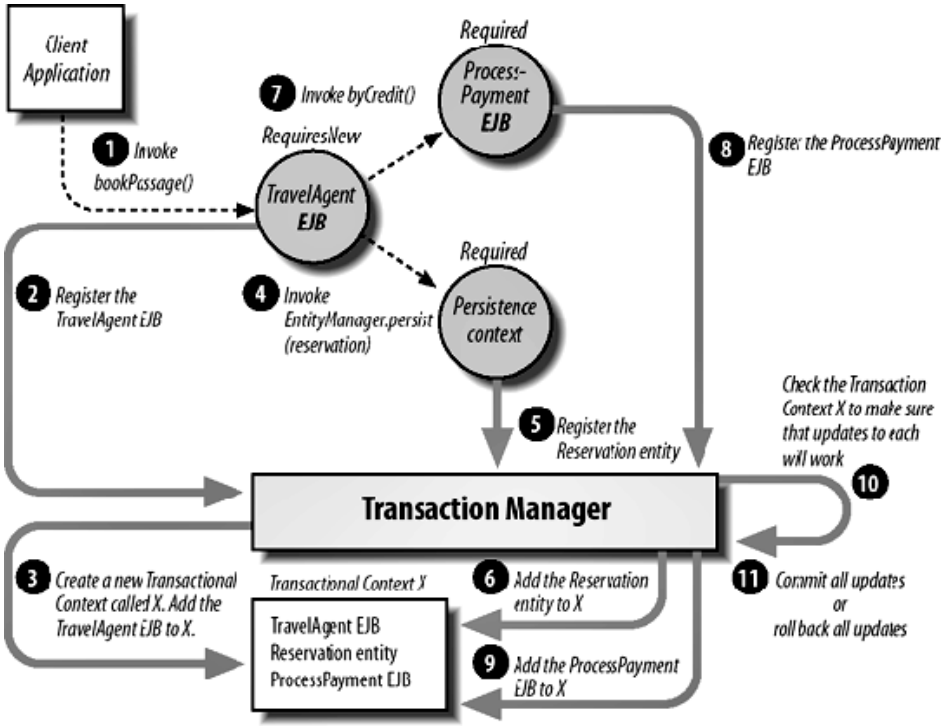
Restrictions

- When a bean method accesses an entity manager, it should have a transaction attribute Required, RequiresNew or Mandatory
- Message-driven beans may declare only the NotSupported or Required attribute

Transaction Propagation

- The EJB container monitors the changes made by the enterprise beans of a transaction scope
- At the end of a transaction the container decides whether to commit or to roll back all the changes
- The EJB container can also coordinate with other transactional systems using the two-phase commit protocol

Transaction Propagation (cont.)



Exceptions and Transactions

System exceptions

- represent unknown internal errors
- are thrown either by the EJB container or the business logic
- are handled automatically by the EJB container
  - roll back the transaction
  - log the exception
  - discard the bean instance
  - throw an EJBTransactionRolledbackException or an EJBException to the client

Exceptions and Transaction (cont.)

Application exceptions

- represent errors in the business logic
- are always delivered directly to the client
- may enforce automatic transaction rollback using the @ApplicationException annotation

```
@ApplicationException(rollback=true)
public class PaymentException extends Exception {
    ...
}
```

Security

Introduction

- Application servers provide security services for authentication, authorization, confidentiality and integrity
- Security services can be integrated declaratively or programmatically into an enterprise bean

Authentication and Identity

- When a remote client logs on to an application server, it is associated with a security identity
- The server implicitly passes the identity with every bean method invocation
- The EJB specification does not specify how authentication happens, many application servers use the JNDI API

```
Properties props = new Properties();
props.put(Context.SECURITY_PRINCIPAL, user);
props.put(Context.SECURITY_CREDENTIALS, password);
InitialContext context = new InitialContext(props);
```


Authorization

- EJB authorization is based on security roles which are mapped to real-world users
- Permissions are assigned to a session bean's methods using annotations
- The EJB container throws an EJBAccessException if a client invokes a method he is not allowed to

Assigning Method Permissions

- The annotation @RolesAllowed defines one or more roles that are allowed to access a method
- The annotation @PermitAll specifies that any authenticated user is allowed to invoke a method
- Annotations placed on the bean class define the default set of permissions that apply to all methods

```
@Stateless
@RolesAllowed("AUTHORIZED_MERCHANT")
public class ProcessPaymentBean implements ProcessPayment {
   @PermitAll
   public boolean byCash(...) throws PaymentException { ... }
   @RolesAllowed("CHECK_FRAUD_ENABLED")
   public boolean byCheck(...) throws PaymentException { ... }
   public boolean byCredit(...) throws PaymentException { ... }
   ...
}
```


The RunAs Security Identity

- The RunAs identity defines the role of an enterprise bean when it tries to invoke methods of another bean

```
@Stateful
@RunAs("AUTHORIZED_MERCHANT")
public class TravelAgentBean implements TravelAgentRemote
{ ... }

@MessageDriven(...)
@RunAs("AUTHORIZED_MERCHANT")
public class ReservationProcessorBean implements MessageListener
{ ... }
```

Programmatic Security

- The session context provides information about the current user of a session bean which can be used for programmatic security checks

```
@Stateless
@DeclareRoles("JUNIOR_AGENT")
public class ProcessPaymentBean implements ProcessPayment {
   @Resource SessionContext context;
   ...
   private boolean process(...) throws PaymentException {
      if (context.isCallerInRole("JUNIOR_AGENT") &&
         amount > maxJuniorTrade) throw new PaymentException(...);
      String agent = context.getCallerPrincipal( ).getName();
      // add travel agent to payment record
      ...
   }
}
```